

Profil menüpont elkészítése

Profil.jsx

A fejlesztés során elkészítettem a Profil menüpontot, amely a bejelentkezett felhasználó adatainak megjelenítésére szolgál. A profil oldal külön React komponensként került létrehozásra. A komponens a localStorage-ban tárolt felhasználói adatokat olvassa ki, amelyek a sikeres bejelentkezést követően kerülnek eltárolásra. Ennek segítségével az aktuálisan bejelentkezett felhasználó adatai jelennek meg a felületen.

```
import "../styles/profil.css";
import { Link } from "react-router-dom";

function Profil() {

  const user = JSON.parse(
    localStorage.getItem("user")
  );

  if (!user) {

    return (

      <div className="profil-container">

        <div className="profil-card">

          <h1>Profil</h1>

          <p>
            Nincs bejelentkezett felhasználó.
          </p>

        </div>

      </div>

    );

  }

  return (

    <div className="profil-container">

      <div className="profil-card">

        <h1>Profilom</h1>
```

```
<div className="adat-sor">

    <span>Név:</span>

    <strong>
        {user.nev || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>E-mail:</span>

    <strong>
        {user.email || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>Telefonszám:</span>

    <strong>
        {user.telefonszam || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>Személyi szám:</span>

    <strong>
        {user.szemelyiszam || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>Lakcím:</span>

    <strong>
        {user.lakcim || "-"}
    </strong>

</div>
```

```
        </div>

        <div className="adat-sor">

            <span>Jelszó:</span>

            <strong>
                {user.jelszo || "-"}
            </strong>

        </div>

        <div className="adat-sor">

            <span>Szerepkör:</span>

            <strong>
                {user.szerepkor || "-"}
            </strong>

        </div>

        <div className="profil-menu">

            <Link
                to="/uzenetek"
                className="profil-gomb"
            >
                Üzeneteim
            </Link>

        </div>

    </div>

</div>

);

}

export default Profil;
```

profil.css

A megjelenés kialakításához külön CSS állományt készítettem. A stíluslap segítségével kártyás megjelenítést, árnyékokat, kiemelt adatmezőket és egységes színvilágot alkalmaztam, amely illeszkedik az alkalmazás többi oldalának megjelenéséhez.

```
.profil-container {  
  
    display: flex;  
    justify-content: center;  
    align-items: center;  
  
    min-height: 80vh;  
}  
  
.profil-card {  
  
    width: 600px;  
  
    background: white;  
  
    border-radius: 12px;  
  
    padding: 40px;  
  
    box-shadow: 0 0 20px rgba(0,0,0,0.2);  
}  
  
.profil-card h1 {  
  
    text-align: center;  
  
    margin-bottom: 30px;  
}  
  
.adat-sor {  
  
    display: flex;  
  
    justify-content: space-between;  
  
    padding: 15px;  
  
    margin-bottom: 10px;  
  
    background-color: #f5f5f5;
```

```
border-left: 5px solid yellow;

border-radius: 5px;
}

.profil-menu {

margin-top: 30px;

display: flex;

justify-content: center;
}

.profil-gomb {

background-color: black;

color: yellow;

text-decoration: none;

padding: 12px 25px;

border-radius: 5px;

font-weight: bold;
}

.profil-gomb:hover {

color: white;
}
```

bejelentkezés.js

A profil oldal a bejelentkezési funkcióhoz kapcsolódik, és a felhasználó adatait jeleníti meg. A felhasználói adatok a /api/bejelentkezés backend végpontról érkeznek sikeres hitelesítés után. A frontend ezeket az adatokat a localStorage tárhelyen tárolja el. A Profil.jsx komponens a localStorage-ból olvassa ki és jeleníti meg az adatokat. Ennek köszönhetően a profil oldal gyorsan betöltődik, külön adatbázis-lekérdezés nélkül.

```
const express = require("express");
```

```
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    email,
    jelszo
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    const users = await conn.query(
      "SELECT * FROM users WHERE email = ? AND jelszo = ?",
      [email, jelszo]
    );

    if (users.length === 0) {

      return res.json({
        siker: false,
        uzenet: "Hibás email vagy jelszó!"
      });

    }

    const user = users[0];

    res.json({
      siker: true,
      uzenet: "Sikeres bejelentkezés!",
      user: {
        nev: user.nev,
        email: user.email,
        jelszo: user.jelszo,
        szerepkor: user.szerepkor,
        létrehozásdatum: user.létrehozásdatum
      }
    });

  } catch (err) {

    console.error(err);

  }

});
```

```
        res.status(500).json({
            siker: false,
            uzenet: "Szerverhiba történt!"
        });

    } finally {

        if (conn) {
            conn.release();
        }

    }

});

module.exports = router;
```

profilmodositas.js

A profil módosításának megvalósításához elkészítettem a megfelelő backend API végpontot. A végpont a felhasználó azonosítója alapján fogadja a módosítandó adatokat, majd feldolgozza azokat. A kérésből kiolvastam a név, e-mail cím, telefonszám, személyi igazolvány szám és lakcím mezőket. Az adatokat SQL UPDATE utasítás segítségével frissítettem az adatbázisban. Ezzel biztosítottam, hogy a felhasználó saját adatait módosítani tudja a rendszerben.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

    const { id } = req.params;

    const {
        nev,
        email,
        telefonszam,
        személyiszam,
        lakcim
    } = req.body;

    let conn;

    try {

        conn = await pool.getConnection();
```

```
    await conn.query(
      `
      UPDATE users
      SET
        nev = ?,
        email = ?,
        telefonszam = ?,
        személyiszam = ?,
        lakcim = ?
      WHERE id = ?
      `,
      [
        nev,
        email,
        telefonszam,
        személyiszam,
        lakcim,
        id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Profil sikeresen módosítva!"
    });

  } catch (err) {

    console.error(err);

    res.status(500).json({
      siker: false,
      uzenet: "Hiba történt!"
    });

  } finally {

    if (conn) {
      conn.release();
    }

  }

});

module.exports = router;
```


profilorlese.js

A felhasználói profil törlésének megvalósításához elkészítettem a megfelelő backend API végpontot. A végpont URL paraméterként fogadja a törlendő felhasználó azonosítóját. Az azonosító alapján megkeresem az adatbázisban a megfelelő rekordot. A művelet végrehajtása után a felhasználó adatai véglegesen eltávolításra kerülnek a rendszerből. Ezzel lehetővé tettem a profilok biztonságos törlését.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

  const { id } = req.params;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `
      DELETE FROM users
      WHERE id = ?
      `,
      [id]
    );

    res.json({
      siker: true,
      uzenet: "Profil sikeresen törölve!"
    });

  } catch (err) {

    console.error(err);

    res.status(500).json({
      siker: false,
      uzenet: "Hiba történt!"
    });

  } finally {

    if (conn) {
```

Szakmai Gyakorlat II. – Frontend – Backend integrációs szakasz

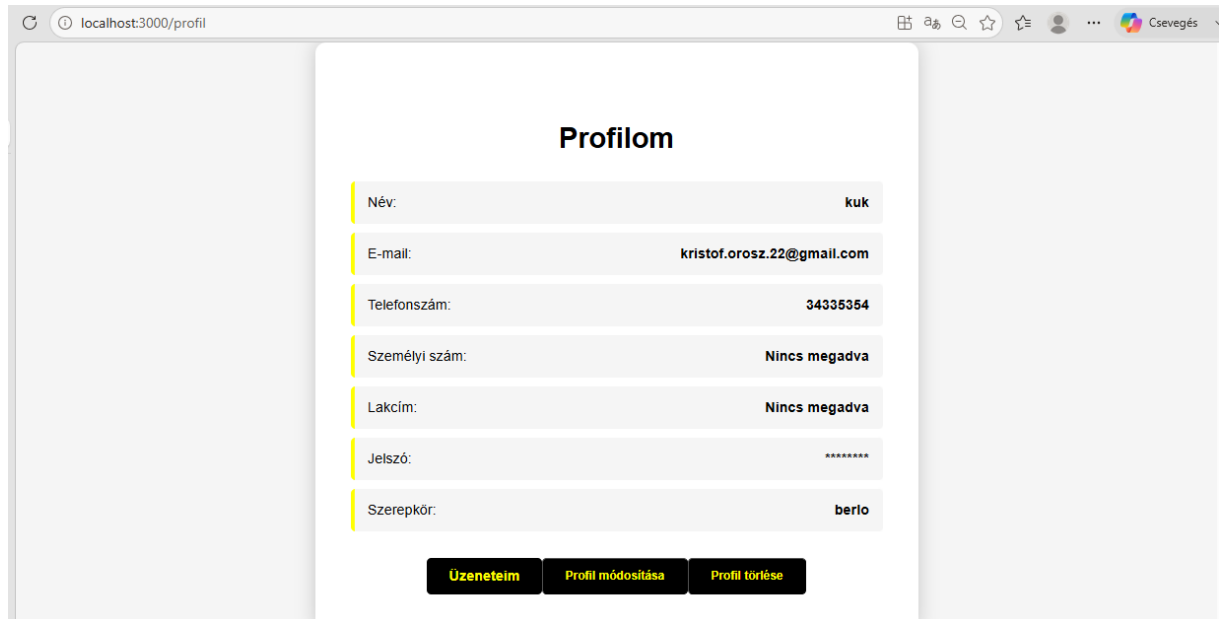
Orosz Kristóf – EYZWG9

```
        conn.release();
    }

}

});

module.exports = router;
```



1.kép: Profil oldal HTML oldala